

Red Hat OpenShift Cluster Architecture

1. Introduction

Red Hat OpenShift is an enterprise-grade Kubernetes platform that simplifies container orchestration by adding opinionated defaults, enhanced security, integrated CI/CD, and developer-friendly tooling. Think of OpenShift as the *thali* of container platforms – Kubernetes is the core dish, while OpenShift adds well-balanced sides like security, networking, storage, and lifecycle management to make it production-ready.

This document explains the architecture of a Red Hat OpenShift cluster, its core components, node roles, networking, storage, and security model.

2. High-Level Architecture Overview

An OpenShift cluster is broadly divided into:

- **Control Plane (Master Nodes)** – Manages the cluster
- **Compute Plane (Worker Nodes)** – Runs user workloads
- **Infrastructure Components** – Networking, storage, registry, ingress, monitoring

At its foundation, OpenShift uses **Kubernetes**, but with Red Hat–supported enhancements and strict security practices.

3. Node Types in OpenShift

3.1 Control Plane Nodes (Master Nodes)

Control plane nodes are responsible for maintaining the desired state of the cluster. These nodes should not run application workloads in production.

Key responsibilities:

- Cluster management
- Scheduling decisions
- API handling
- State persistence

Core components running on control plane nodes:

a) kube-apiserver

- Central entry point to the cluster
- All `oc` and `kubectl` commands communicate with the API server
- Validates and processes REST requests

b) etcd

- Distributed key-value store
- Stores cluster state, configuration, secrets, and metadata
- Requires high availability (odd number of nodes: 3, 5, etc.)

c) kube-scheduler

- Assigns pods to suitable worker nodes
- Considers resource availability, affinity rules, and constraints

d) kube-controller-manager

- Runs controllers that maintain cluster state
- Examples: node controller, replication controller, endpoint controller

e) OpenShift Controller Manager

- Manages OpenShift-specific resources like Routes, Projects, and Builds
-

3.2 Worker Nodes (Compute Nodes)

Worker nodes are where application workloads run.

Key responsibilities:

- Run application pods
- Provide compute, memory, and networking

Core components on worker nodes:

a) kubelet

- Agent running on each node
- Communicates with the API server
- Ensures containers are running as defined in pod specs

b) Container Runtime

- CRI-O (default in OpenShift)
- Responsible for running containers

c) kube-proxy

- Handles network rules
 - Enables service-to-pod communication
-

3.3 Infrastructure Nodes (Optional)

In larger clusters, dedicated infrastructure nodes are used for:

- Load balancers
- Container image registry
- Monitoring and logging stacks

This separation improves performance and security, similar to keeping the *prasadam* area separate from the main kitchen during a temple festival.

4. OpenShift Networking Architecture

OpenShift uses a **Software Defined Networking (SDN)** model.

4.1 Cluster Network

- Each pod gets a unique IP address
- Pods can communicate across nodes without NAT

4.2 Service Network

- Services provide stable virtual IPs
- Abstract pod IP changes

4.3 Network Plugins

Common CNI options:

- OVN-Kubernetes (default in newer OpenShift versions)
- OpenShift SDN (older versions)

4.4 Ingress and Routes

- OpenShift uses **Routes** instead of plain Kubernetes Ingress
 - HAProxy-based routers expose services externally
-

5. Storage Architecture

OpenShift supports both ephemeral and persistent storage.

5.1 Persistent Volumes (PV) and Claims (PVC)

- Applications request storage using PVCs
- Cluster admins provide PVs

5.2 Supported Storage Types

- NFS
- iSCSI
- Ceph (Red Hat OpenShift Data Foundation)
- Cloud storage (EBS, Azure Disk, GCP PD)

5.3 CSI (Container Storage Interface)

- OpenShift uses CSI drivers
 - Enables dynamic provisioning and snapshots
-

6. Security Architecture

Security is a key differentiator of OpenShift.

6.1 Authentication and Authorization

- Integrates with LDAP, OAuth, GitHub, GitLab, and Red Hat SSO
- Uses **RBAC (Role-Based Access Control)**

6.2 Security Context Constraints (SCC)

- OpenShift-specific security feature
- Controls:
 - Running as non-root
 - Capabilities
 - SELinux contexts

This is stricter than upstream Kubernetes, much like Indian traffic rules becoming stricter near school zones.

6.3 SELinux Integration

- Enforced by default
 - Provides mandatory access control
-

7. Image Registry Architecture

OpenShift includes an internal container registry:

- Stores application images
- Integrated with authentication and RBAC
- Can be backed by persistent storage

Supports:

- ImageStreams
 - Automatic image rebuilds on updates
-

8. Monitoring and Logging

8.1 Monitoring

- Prometheus for metrics collection
- Alertmanager for alerts
- Grafana for visualization

8.2 Logging

- EFK stack (Elasticsearch, Fluentd, Kibana) or Loki
 - Centralized log management
-

9. CI/CD and Developer Workflow

OpenShift integrates CI/CD tools:

- OpenShift Pipelines (Tekton)
- OpenShift GitOps (Argo CD)

Developer workflow:

1. Code commit
 2. BuildConfig or Pipeline triggers build
 3. Image stored in registry
 4. Deployment to cluster
-

10. High Availability and Scalability

- Control plane runs in HA mode
- Worker nodes can be scaled horizontally
- Self-healing through Kubernetes controllers

This resilience mirrors Indian railways – delays may happen, but alternate routes keep things moving.

11. Summary

Red Hat OpenShift provides a robust, secure, and enterprise-ready Kubernetes platform by combining:

- Kubernetes core
- Strong security defaults
- Integrated networking, storage, and CI/CD

Understanding this architecture is essential for administrators, developers, and DevOps engineers preparing for real-world production clusters and certifications.

Document prepared for training and architectural understanding of Red Hat OpenShift clusters.

12. Logical Architecture View (How Components Fit Together)

To visualize how all OpenShift components interact, think of the cluster as a layered system:

Layer 1: User & Developer Access Layer

- Web Console (Browser UI)
- oc / kubectl CLI
- CI/CD systems (Jenkins, Tekton, GitHub Actions)

All requests from users and automation tools first reach the **kube-apiserver**.

Layer 2: Control Plane Layer

This is the brain of the cluster and runs on **control plane (master) nodes**.

Components:

- kube-apiserver – Entry point for all operations
- etcd – Stores entire cluster state
- kube-scheduler – Selects worker nodes for pods
- kube-controller-manager – Ensures desired state
- OpenShift Controller Manager – Manages Routes, Projects, Builds

Flow:

User → API Server → etcd / Controllers / Scheduler

Layer 3: Worker Node Layer (Compute Plane)

Where applications actually run.

Each worker node contains:

- kubelet – Talks to API server and manages pods
- CRI-O – Runs containers
- kube-proxy – Implements service networking
- Pods – One or more containers

Flow:

Scheduler → Worker Node → kubelet → CRI-O → Pod

Layer 4: Networking Layer

Provides pod-to-pod and service connectivity.

Components:

- OVN-Kubernetes (default CNI)
- Services (ClusterIP, NodePort, LoadBalancer)
- Routes (OpenShift-specific external access)
- Ingress Controller (HAProxy)

Flow:

User → Route → Ingress Router → Service → Pod

Layer 5: Storage Layer

Provides persistent data for applications.

Components:

- PersistentVolume (PV)

- PersistentVolumeClaim (PVC)
- CSI drivers
- OpenShift Data Foundation (Ceph)

Flow:

Pod → PVC → CSI → Storage Backend

Layer 6: Security Layer

Enforces secure defaults across the cluster.

Components:

- RBAC – User authorization
- SCC – Pod-level security
- SELinux – Mandatory access control
- OAuth – Authentication

Flow:

User → OAuth → API Server → RBAC/SCC → Resource Access

Layer 7: Platform Services Layer

Built-in services that make OpenShift enterprise-ready.

Components:

- Internal Image Registry
 - ImageStreams
 - Monitoring (Prometheus, Grafana)
 - Logging (Loki / EFK)
 - Pipelines (Tekton)
 - GitOps (Argo CD)
-

13. End-to-End Request Flow Example

Example: Deploying an application

1. Developer runs: `oc apply -f app.yaml`
2. Request reaches kube-apiserver
3. API server validates and stores data in etcd
4. Scheduler assigns pod to a worker node
5. kubelet creates the pod via CRI-O

6. Image pulled from internal registry
7. Pod gets IP via OVN-Kubernetes
8. Service exposes pod internally
9. Route exposes service externally

14. Why This Architecture Matters

This layered and modular architecture provides:

- High availability
- Horizontal scalability
- Strong security defaults
- Clear separation of concerns
- Enterprise-grade reliability

It is this design that makes OpenShift suitable for regulated, high-scale production environments.

